

■ KALI → BOSS OS

Bug Bounty Programme 2026

The Vibe Coder's Hunting Guide

PS-02 · PS-07 · PS-08 — Complete Preparation Guide for Midhilesh

You don't write code. You direct AI to write it. That is your superpower.

3	Kali	AI
Problem Statements	Linux (Already Installed!)	Powered Workflow

Team ZeroDay · July 7–8, 2026 · Chennai Venue

You are the team's automation engine. Every AI prompt in this guide is ready to copy-paste.

Table of Contents

Chapter 1	Your Role — The Vibe Coder Strategy
Chapter 2	Your Weapon — Kali Linux on VirtualBox
Chapter 3	PS-02: Authentication, Access Control & Privilege
Chapter 4	PS-07: File System, Permissions & Storage
Chapter 5	PS-08: Logging, Auditing & Monitoring
Chapter 6	7-Day Preparation Plan
Chapter 7	Competition Day — First 30 Minutes
Chapter 8	Bug Report Template + Real Examples
Chapter 9	Master Cheat Sheet

CHAPTER 1

Your Role — The Vibe Coder Strategy

You don't need to memorize commands. You need to know what to ask.

What Is a Vibe Coder in Bug Bounty?

A vibe coder is someone who understands WHAT needs to be done and uses AI to figure out HOW to do it. In a bug bounty, this means you know what vulnerabilities to look for, and you ask AI to write the scripts that find them. The AI writes code. You direct it. You review findings. You write reports.

★ The best bug bounty hunters are not the best coders. They are the best THINKERS who know what to look for. You are already that person.

Team ZeroDay — Full Division

Member	Focus	Weapon
Darshini	PS-01: Kernel Security	Theoretical knowledge + tools
Mageshwaran	PS-03, PS-04, PS-09: Network/Crypto/Pkgs	Python scripting
Midhilesh	PS-02, PS-07, PS-08: Auth/Files/Logs	Kali Linux + AI prompting

The Vibe Coder Workflow

Every time you need to do something in the competition, follow this loop:

Step	What You Do	Example
1. Know the goal	Understand WHAT you are looking for	Find weak file permissions
2. Ask AI	Give AI the exact context + task	Prompt from this guide
3. Run the script	Copy-paste into Kali terminal, execute	python3 script.py
4. Read results	Understand what the output means	Flag = vulnerability?
5. Write report	Document the finding with proof	Use Chapter 8 template
6. Repeat	Move to next check	Stay systematic

■ Every chapter in this guide has YELLOW AI PROMPT BOXES. These are ready-to-use prompts you copy into Claude or ChatGPT during the competition to instantly get the script you need. No memorization required.

CHAPTER 2

Your Weapon — Kali Linux

You already have it. Here is how to use it as your attack base.

What You Already Have

You already have VirtualBox with Kali Linux installed on your laptop. Kali is the world's most powerful security testing operating system — used by professional hackers and security researchers. It comes with 600+ tools pre-installed.

■ Your setup: Laptop → VirtualBox → Kali Linux (attack machine) → BOSS OS VM (target). You will run tools FROM Kali AGAINST the BOSS OS VM. This is the exact same setup professionals use.

Setting Up the Dual VM Environment

S t e p	What To Do
1	Open VirtualBox — you will see your Kali Linux VM already there
2	When BOSS OS 11.0 is released, download the .iso or .ova file
3	In VirtualBox: click New → import the BOSS OS VM
4	Set BOTH VMs to use 'Host-Only Adapter' network (so they can talk to each other)
5	Start BOSS OS VM → note its IP: run 'ip addr' inside BOSS OS
6	Start Kali VM → you can now scan BOSS OS from Kali
7	Test connection: ping <BOSS_OS_IP> from Kali terminal

Essential Kali Commands to Know

```
# Find the IP of BOSS OS (run this inside BOSS OS terminal)
ip addr show
hostname -I

# From Kali – verify you can reach BOSS OS
ping <BOSS_OS_IP>

# Open a terminal in Kali
# Right click desktop → Open Terminal
# OR press Ctrl+Alt+T
# Check Kali version
cat /etc/os-release

# Update Kali tools (do this on Day 1)
sudo apt update
```

Important: Two Modes of Testing

Mode	What It Means	When To Use
Local testing (inside BOSS OS)	Log into BOSS OS directly and run commands there	PS-07, PS-08 — checking files and logs
Remote testing (Kali → BOSS OS)	Run tools FROM Kali targeting BOSS OS IP	PS-02 — testing login/auth from outside

CHAPTER 3

PS-02: Authentication, Access Control & Privilege

Finding weaknesses in how BOSS OS verifies who you are and what you can do.

PS-02**Authentication, Access Control, and Privilege Management**

What Does This Mean? (Plain English)

Authentication means 'proving who you are' — like logging in with a password. Access control means 'deciding what you can do after logging in'. Privilege management means 'making sure normal users cannot do admin things'.

Think of a government building. Authentication = the guard checking your ID. Access control = which rooms your ID badge opens. Privilege = whether you can access the server room. If any of these are weak, an attacker gets in.

What Bugs Exist Here?

Bug	What It Means	Severity
Default/weak passwords	Accounts using 'admin', '1234', 'boss' as password	CRITICAL
Empty passwords	Accounts with NO password at all	CRITICAL
Passwordless sudo	User can run admin commands without password	CRITICAL
World-readable /etc/shadow	Password hashes visible to everyone	CRITICAL
Excessive sudo privileges	Normal users can run too many admin commands	HIGH
No account lockout	Unlimited login attempts allowed (brute-force risk)	HIGH
Inactive root account open	Root login enabled and not locked	HIGH
Weak PAM config	Password policy allows short/simple passwords	MEDIUM
Users in wrong groups	Normal user is in 'sudo' or 'wheel' group by mistake	HIGH
Unused accounts active	Old accounts still enabled and accessible	MEDIUM

Check 1 — User Accounts & Passwords

```
# Run these INSIDE BOSS OS terminal
# List all user accounts
cat /etc/passwd

# Check which accounts have NO password (! or * = locked, empty = NO
PASSWORD = CRITICAL)
sudo cat /etc/shadow | awk -F: '($2==" " || $2=="!") {print $1" has NO
password!"}'

# Check if /etc/shadow is readable (CRITICAL if yes)
cat /etc/shadow 2>/dev/null && echo 'CRITICAL: shadow readable!'

# List all users who can log in (have a shell)
grep -v '/nologin\|/false' /etc/passwd | cut -d: -f1

# Check password expiry policies
chage -l <username>
```

Check 2 — Sudo Privileges

```
# Check current user's sudo permissions
sudo -l

# Check the sudoers file (who can run what as root)
cat /etc/sudoers 2>/dev/null
ls /etc/sudoers.d/

# DANGEROUS patterns to look for:
# 'NOPASSWD: ALL' = user can do ANYTHING as root without password =
CRITICAL
# 'ALL=(ALL) ALL' = user has full sudo = check if this user should have it

# Check who is in the sudo group
getent group sudo
getent group wheel
getent group admin
```

■ AI PROMPT — Copy this into Claude/ChatGPT

I am doing a security audit of BOSS OS (Debian-based Linux). Write me a Python script that: 1. Reads /etc/passwd and lists all accounts that have a login shell 2. Tries to read /etc/shadow and reports if it is readable (vulnerability) 3. Checks /etc/sudoers for NOPASSWD entries and reports them 4. Checks which users are in the sudo, wheel, and admin groups 5. Outputs a clear report with CRITICAL/HIGH/MEDIUM labels for each finding Make it beginner friendly with clear print statements.

Check 3 — Password Policy

```
# Check PAM password policy (how complex passwords must be)
cat /etc/pam.d/common-password
# Look for these settings:
# minlen=8 means minimum 8 characters (should be at least 12)
# ucredit=-1 means must have uppercase (good)
# If NO complexity rules exist = MEDIUM vulnerability
# Check login attempt limits (brute force protection)
cat /etc/pam.d/common-auth | grep pam_tally
cat /etc/pam.d/common-auth | grep pam_faillock
# If neither exists = no lockout policy = HIGH vulnerability
# Check password minimum age
grep 'PASS_MIN_DAYS\|PASS_MAX_DAYS\|PASS_WARN_AGE' /etc/login.defs
```

■ AI PROMPT — Copy this into Claude/ChatGPT

I am auditing BOSS OS (Debian Linux) for PS-02 Authentication security. Write a bash script that checks: 1. /etc/pam.d/common-password for minimum password length and complexity rules 2. /etc/pam.d/common-auth for account lockout settings (pam_tally2 or pam_faillock) 3. /etc/login.defs for PASS_MIN_DAYS, PASS_MAX_DAYS, PASS_WARN_AGE values 4. Whether root account is locked (passwd -S root) For each check, print whether it PASSES or FAILS security best practice, and what the current value is. Label each finding with severity.

Check 4 — From Kali: Test SSH Authentication

Run these FROM your Kali Linux terminal targeting the BOSS OS IP:

```
# Replace <BOSS_IP> with the actual IP of the BOSS OS VM
# Try connecting as root (should be blocked)
ssh root<BOSS_IP>
# If you get a password prompt instead of 'Permission denied' = HIGH bug
# Try common default passwords (ethical testing only)
ssh boss<BOSS_IP> # try password: boss
ssh admin<BOSS_IP> # try password: admin
ssh user<BOSS_IP> # try password: user
# Scan SSH version and config from outside
nmap -sV -p 22 <BOSS_IP>
nmap --script ssh-auth-methods -p 22 <BOSS_IP>
```

CHAPTER 4

PS-07: File System, Permissions & Storage

Finding files and folders that are accessible to the wrong people.

PS-07 File System, Permissions, and Storage**What Does This Mean? (Plain English)**

Every file and folder on Linux has permissions — who can read it, write to it, or run it. Think of files as documents in a filing cabinet. Some are public, some are private, some are top secret. If the wrong person can access a top secret file, that is a vulnerability.

Linux Permission System — Quick Explanation

Symbol	Means	Example
r	Read — can see the contents	-rw-r--r-- (owner can read+write, others read)
w	Write — can modify/delete	-rw-rw---- (owner+group can modify)
x	Execute — can run as program	-rwxr-xr-x (everyone can run)
s	SUID/SGID — runs as owner's privileges	-rwsr-xr-x (runs as ROOT!)
777	World readable+writable+executable	Anyone can do anything = BIG BUG
644	Owner rw, others read only	Normal file permission
600	Only owner can read/write	Good for private files

What Bugs Exist in File Permissions?

Bug	What It Means	Severity
World-writable system files	Anyone can modify /etc/passwd or system configs	CRITICAL
Unneeded SUID binaries	Files that run as root unnecessarily	HIGH
Sensitive files world-readable	Config files with passwords visible to all	HIGH
Writable /etc/cron.d	Anyone can schedule root commands	CRITICAL
World-writable /tmp misuse	/tmp writable but sticky bit missing	MEDIUM
Sensitive backups readable	.bak files with passwords accessible	HIGH
Private keys accessible	SSH keys or SSL certs readable by normal user	CRITICAL
Log files world-writable	Anyone can tamper with security logs	HIGH

Check 1 — SUID & SGID Files

```
# Find ALL files that run with elevated privileges
find / -perm -4000 -type f 2>/dev/null | tee suid_files.txt
# Find SGID files (run with group privileges)
find / -perm -2000 -type f 2>/dev/null | tee sgid_files.txt
# For each result, check if it should have SUID
# Common LEGITIMATE SUID: passwd, sudo, su, ping, mount
# SUSPICIOUS: any custom script or unusual binary with SUID
# Check a specific SUID file's details
ls -la /usr/bin/passwd
stat /usr/bin/passwd
```

Check 2 — World-Writable Files & Directories

```
# Find world-writable files in system directories (BIG BUG if found)
find /etc /usr /bin /sbin /lib -perm -o+w -type f 2>/dev/null
# Find world-writable directories (excluding /tmp, /proc, /sys)
find / -perm -o+w -type d 2>/dev/null | grep -v
'^/proc\|^/sys\|^/tmp\|^/dev'
# Check /tmp has sticky bit (t flag at end = correct)
ls -la / | grep tmp
# Should show: drwxrwxrwt (the 't' at end = sticky bit = correct)
# If shows: drwxrwxrwx (no 't') = MEDIUM vulnerability
# Find cron directories that are writable
ls -la /etc/cron* /var/spool/cron/ 2>/dev/null
```

■ AI PROMPT — Copy this into Claude/ChatGPT

I am auditing BOSS OS file system permissions for a bug bounty competition. Write a Python script that: 1. Finds all SUID files (perm -4000) outside of /proc and /sys 2. Finds all world-writable files in /etc, /usr, /bin, /sbin 3. Checks if /etc/passwd is world-writable (CRITICAL if yes) 4. Checks if /etc/shadow is world-readable (CRITICAL if yes) 5. Finds any .bak, .backup, .old files in /etc that might contain passwords 6. Checks /tmp for missing sticky bit Output each finding with severity (CRITICAL/HIGH/MEDIUM) and the exact file path.

Check 3 — Sensitive File Exposure

```
# Check permissions on critical system files
ls -la /etc/passwd /etc/shadow /etc/group /etc/gshadow
# /etc/passwd should be: -rw-r--r-- (world readable = OK, it has no
passwords)
# /etc/shadow should be: -rw-r----- or -rw----- (NOT world readable)
# Find backup files that might contain credentials
find / -name '*.bak' -o -name '*.backup' -o -name '*.old' -o -name
 '*.orig' 2>/dev/null
# Find private SSH keys that should NOT be world-readable
find / -name 'id_rsa' -o -name 'id_ed25519' -o -name '*.pem' 2>/dev/null
# Find config files with 'password' in them (may expose credentials)
grep -r 'password\s*=' /etc/ 2>/dev/null | grep -v '#'
```

■ AI PROMPT — Copy this into Claude/ChatGPT

I am doing a file system security audit on BOSS OS for PS-07. Write a bash script that: 1. Searches /etc and /home for files containing the word 'password' or 'passwd' 2. Finds .bak .old .orig .backup files anywhere on the system 3. Checks permissions on /etc/passwd, /etc/shadow, /etc/crontab, /etc/sudoers 4. Finds any SSH private key files (id_rsa, id_ed25519, *.pem) and checks if they are world-readable 5. Reports each finding clearly with the file path and why it is a problem. Keep it simple and beginner-friendly.

CHAPTER 5

PS-08: Logging, Auditing & Monitoring

Finding weaknesses in how BOSS OS records and monitors what happens.

PS-08 Logging, Auditing, and Monitoring**What Does This Mean? (Plain English)**

Logging means writing a record of everything that happens — who logged in, what commands they ran, what files they accessed, when errors occurred. Auditing means reviewing those records. Monitoring means watching for suspicious activity in real time.

If a government OS does not log properly, attackers can do things without leaving a trace. If logs are writable by anyone, attackers can erase evidence. Your job is to find these weaknesses.

What Bugs Exist in Logging?

Bug	What It Means	Severity
No audit daemon (auditd)	System not recording security events at all	CRITICAL
World-writable log files	Anyone can modify or erase security logs	CRITICAL
Logs not capturing sudo use	Admin actions not being recorded	HIGH
Failed login not logged	Brute force attacks leave no trace	HIGH
Log rotation misconfigured	Logs fill up disk or get deleted too fast	MEDIUM
Syslog not running	System-wide logging service is off	HIGH
No remote log backup	Logs only stored locally, easily deleted	MEDIUM
Audit rules too permissive	auditd not watching critical files/commands	MEDIUM
Cron logs disabled	Scheduled tasks not being recorded	MEDIUM

Check 1 — Is auditd Running?

auditd is the Linux kernel audit daemon. It **MUST** be running on a government OS. If it is not — that is a CRITICAL vulnerability.

```
# Check if auditd is installed and running
systemctl status auditd

# If output shows 'inactive' or 'not found' = CRITICAL BUG
# If output shows 'active (running)' = good, check its rules next

# Check what auditd is monitoring
auditctl -l 2>/dev/null
cat /etc/audit/audit.rules 2>/dev/null
cat /etc/audit/rules.d/*.rules 2>/dev/null

# Check recent audit logs
ausearch -ts recent 2>/dev/null | head -20
cat /var/log/audit/audit.log 2>/dev/null | head -20
```

Check 2 — Log File Permissions

```
# Check permissions on ALL log files
ls -la /var/log/

# CRITICAL: Any log file that is world-writable
find /var/log -perm -o+w -type f 2>/dev/null

# If ANY result = CRITICAL (anyone can tamper with logs)

# Check auth log (records all login attempts)
ls -la /var/log/auth.log
cat /var/log/auth.log | tail -20

# Check syslog
ls -la /var/log/syslog

# Check if important logs even exist
ls /var/log/auth.log /var/log/syslog /var/log/kern.log 2>/dev/null

# Missing files = logging not configured = HIGH vulnerability
```

■ AI PROMPT — Copy this into Claude/ChatGPT

I am auditing BOSS OS logging and monitoring for PS-08 bug bounty. Write a Python script that: 1. Checks if auditd service is running using subprocess 2. Checks if /var/log/auth.log, /var/log/syslog, /var/log/kern.log exist 3. Finds any world-writable files in /var/log/ directory 4. Checks if /etc/audit/audit.rules exists and has content 5. Reads the last 10 lines of /var/log/auth.log and checks for failed login entries 6. Checks if rsyslog or syslog service is running For each check, label the result as PASS, FAIL-CRITICAL, FAIL-HIGH, or FAIL-MEDIUM with explanation of why it matters.

Check 3 — Are Critical Actions Being Logged?

```
# Check if sudo commands are being logged
grep 'sudo' /var/log/auth.log 2>/dev/null | tail -5
grep 'COMMAND' /var/log/auth.log 2>/dev/null | tail -5
# Check if failed logins are logged
grep 'Failed\\|FAILED\\|failure' /var/log/auth.log 2>/dev/null | tail -5
# If these return nothing = failed logins not logged = HIGH bug
# Check rsyslog configuration
cat /etc/rsyslog.conf 2>/dev/null | grep -v '#' | grep -v '^$'
systemctl status rsyslog
# Check if cron logging is enabled
grep -i 'cron' /etc/rsyslog.conf 2>/dev/null
ls -la /var/log/cron* /var/log/syslog 2>/dev/null
# Check log rotation config
cat /etc/logrotate.conf | head -20
ls /etc/logrotate.d/
```

Check 4 — Audit Rules Coverage

If auditd IS running, check if it is monitoring the RIGHT things:

```
# Check current audit rules
sudo auditctl -l

# A well-configured system should monitor:
# /etc/passwd and /etc/shadow changes
# /etc/sudoers changes
# All commands run by root
# Failed authentication attempts
# If none of these are in the rules = MEDIUM vulnerability
# Example of what good rules look like:
# -w /etc/passwd -p wa -k identity
# -w /etc/shadow -p wa -k identity
# -w /etc/sudoers -p wa -k sudo_changes
```

■ AI PROMPT — Copy this into Claude/ChatGPT

I am checking audit rules on BOSS OS for a security competition. Write a bash script that: 1. Runs 'auditctl -l' and checks if rules exist for /etc/passwd, /etc/shadow, /etc/sudoers 2. Checks if there are rules for monitoring privileged commands (execve syscall) 3. Checks if audit log file at /var/log/audit/audit.log exists and is not empty 4. Checks if the audit log directory /var/log/audit has correct permissions (should not be world-readable) 5. Reports each missing audit rule as a finding with MEDIUM severity and explains what attacker activity would go undetected without it.

CHAPTER 6

7-Day Preparation Plan

One day at a time. Use AI for every script. Stay systematic.

■ Your prep is different from Darshini and Mageshwaran. You spend less time memorizing and more time **PRACTICING** the vibe coder workflow — prompt → script → run → report. Do that loop every day.

Day 1 — Environment & Kali Familiarization

■ Goal: Get Kali running and connecting to a test Linux VM

1. Open VirtualBox and start your Kali Linux VM
2. Open a terminal in Kali (right click desktop or Ctrl+Alt+T)
3. Run: `uname -a` (check Kali version)
4. Run: `sudo apt update` (update all tools)
5. Create an Ubuntu or Debian VM (the practice target — same as BOSS OS base)
6. Set both VMs to Host-Only Network in VirtualBox settings
7. From Kali: ping the Ubuntu VM's IP to confirm they can communicate
8. Take a snapshot of both clean VMs
9. Success: Kali can reach the target VM. You can open a terminal.

Day 2 — PS-02 Authentication Hunting

■ Goal: Find at least 3 authentication issues in your target VM

1. Log into your Ubuntu/target VM (not Kali)
2. Run all Check 1 commands from Chapter 3 (user accounts and passwords)
3. Run all Check 2 commands from Chapter 3 (sudo privileges)
4. Use AI Prompt 1 from Chapter 3 — generate the Python script, run it
5. From Kali: run `'nmap --script ssh-auth-methods -p 22 '`
6. Try SSHing as root from Kali to the target (should fail — if not, bug!)
7. Document every finding: what it is, severity, how you found it
8. Success: You have a written list of at least 3 authentication findings

Day 3 — PS-07 File Permission Hunting

■ Goal: Find SUID files and world-writable misconfigs

1. Log into your target VM
2. Run: `find / -perm -4000 -type f 2>/dev/null` (save to `suid.txt`)
3. For each SUID file found, Google 'is [filename] supposed to be SUID'
4. Run: `find /etc -perm -o+w -type f 2>/dev/null` (world-writable system files)
5. Use AI Prompt from Chapter 4 — generate Python script, run it

6. Check: `ls -la /etc/shadow` (what are the permissions?)
7. Check: `find / -name '*.bak' 2>/dev/null` (backup files with credentials?)
8. Write a bug report for the most interesting finding
9. Success: You understand file permissions and found real issues

Day 4 — PS-08 Logging Audit

■ Goal: Check if the system is properly recording security events

1. Log into target VM and run: `systemctl status auditd`
2. Run: `ls -la /var/log/` (what logs exist and what are their permissions?)
3. Run: `find /var/log -perm -o+w 2>/dev/null` (world-writable logs = bug!)
4. Use AI Prompt 1 from Chapter 5 — generate and run the Python audit script
5. Check rsyslog: `systemctl status rsyslog`
6. Deliberately cause a failed login: `ssh wronguser@localhost`
7. Check if it was logged: `grep 'Failed' /var/log/auth.log`
8. Document findings with evidence (copy the exact command + output)
9. Success: You know what is and is not being logged

Day 5 — LinPEAS + Cross-Domain Review

■ Goal: Use LinPEAS to find bugs you missed manually

1. Download LinPEAS on target: `wget [linpeas URL] && chmod +x linpeas.sh`
2. Run LinPEAS and save output: `./linpeas.sh | tee linpeas.txt`
3. Read through RED sections — each is a potential finding
4. Cross-reference LinPEAS results with your Day 2-4 manual findings
5. For any new findings LinPEAS shows, investigate and document
6. Practice writing 2 full bug reports using the Chapter 8 template
7. Ask AI: 'Review this bug report and suggest improvements' (paste your report)
8. Success: You have 5+ documented findings and 2 complete bug reports

Day 6 — 30-Minute Speed Competition Drill

■ Goal: Complete a full audit of all 3 domains in 30 minutes

1. Restore your CLEAN target VM snapshot
2. Set timer: 30 minutes
3. Run in this order: LinPEAS → PS-02 checks → PS-07 checks → PS-08 checks
4. Use your AI prompts to generate scripts instantly if needed
5. Write a 3-line summary for each finding as you go (no stopping)
6. After 30 mins: count findings. Aim for 5+ across the 3 domains
7. Repeat the drill — try to be 5 minutes faster the second time
8. Success: You complete the basic audit in under 25 minutes

Day 7 — Rest, Prepare & Coordinate

■ Goal: Be fresh, organised and ready

1. Review all your notes and findings from the week
2. Prepare your personal cheat sheet (use Chapter 9 as base)
3. Save all your AI prompts somewhere easy to access during competition
4. Coordinate with Darshini and Mageshwaran — confirm everyone's domains
5. Make sure VirtualBox is working and Kali VM starts cleanly
6. Do NOT study new things — rest your brain
7. Sleep early the night before. Eat well. Bring water and snacks.
8. Success: You feel confident. You know exactly what to do and when.

CHAPTER 7

Competition Day — First 30 Minutes

Your minute-by-minute action plan. Follow this exactly.

■ You have Kali Linux. You have AI prompts. You have 3 domains. Launch automated tools first, investigate results second. Never the other way.

Minute 0–3: Boot and Connect

```
# Start both VMs: Kali + BOSS OS
# Get BOSS OS IP (run inside BOSS OS terminal)
ip addr show | grep 'inet '
# From Kali — confirm connection
ping -c 3 <BOSS_OS_IP>
# Also open a terminal INSIDE BOSS OS for local checks
# You will be switching between both terminals
```

Minute 3–8: Launch All Automated Tools

```
# INSIDE BOSS OS terminal — launch LinPEAS in background
wget -q https://github.com/carlospolop/PEASS-ng/releases/latest/download/
linpeas.sh
chmod +x linpeas.sh
./linpeas.sh | tee linpeas.txt &
# FROM KALI terminal — scan BOSS OS from outside
nmap -sV -p 22,80,443,21,23,25 <BOSS_OS_IP> | tee nmap_results.txt &
nmap --script ssh-auth-methods -p 22 <BOSS_OS_IP> >> nmap_results.txt &
```

Minute 8–15: PS-02 Authentication Checks

```
# INSIDE BOSS OS
cat /etc/shadow 2>/dev/null && echo 'CRITICAL: shadow readable!'
sudo -l
grep 'NOPASSWD' /etc/sudoers /etc/sudoers.d/* 2>/dev/null
getent group sudo wheel admin
grep -v '/nologin\|/false' /etc/passwd | cut -d: -f1
# FROM KALI
ssh root@<BOSS_OS_IP> # Should be rejected
ssh admin@<BOSS_OS_IP> # Try default password: admin
ssh boss@<BOSS_OS_IP> # Try default password: boss
```

Minute 15–22: PS-07 File Permission Checks

```
# INSIDE BOSS OS
find / -perm -4000 -type f 2>/dev/null | tee suid.txt
find /etc /usr /bin -perm -o+w -type f 2>/dev/null | tee
world_writable.txt
ls -la /etc/shadow /etc/passwd /etc/sudoers
find / -name '*.bak' -o -name '*.old' 2>/dev/null | tee backups.txt
find / -name 'id_rsa' -o -name '*.pem' 2>/dev/null | tee keys.txt
ls -la / | grep tmp # Check for sticky bit 't'
```

Minute 22–30: PS-08 Logging Checks

```
# INSIDE BOSS OS
systemctl status auditd rsyslog syslog 2>/dev/null
ls -la /var/log/
find /var/log -perm -o+w 2>/dev/null | tee writable_logs.txt
cat /var/log/auth.log 2>/dev/null | tail -10
auditctl -l 2>/dev/null
grep 'Failed\|FAILED' /var/log/auth.log 2>/dev/null | wc -l
# If auditd is not running = REPORT IMMEDIATELY
# If /var/log/auth.log does not exist = REPORT IMMEDIATELY
```

Minute 30+: Review LinPEAS + Write Reports

```
# LinPEAS should be done now – review RED sections
grep -A3 'CVE\|CRITICAL\|Interesting' linpeas.txt | head -40
# Start writing reports IMMEDIATELY for each finding
# Do NOT wait until you have found everything
# Submit reports as you go
# If you need a script quickly – use your AI prompts from this guide
# Copy the prompt into Claude on your phone or another tab
# Paste and run the generated script immediately
```

CHAPTER 8

Bug Report Template + Real Examples

One template. Use it for every single bug you find.

■ Judges can only score what you write. A clearly documented MEDIUM bug earns more points than a vaguely described CRITICAL one. Write clearly.

Vulnerability Title

Specific + includes component + what is wrong.

GOOD: 'No Account Lockout Policy — Unlimited SSH Brute Force Attempts Allowed'

GOOD: '/var/log/auth.log is World-Writable — Attackers Can Erase Login Evidence'

BAD: 'Found a bug in the logs'

BOSS OS Version

Run: `uname -a` inside BOSS OS and paste the full output.

Example: Linux boss-desktop 5.15.0-1-boss #1 SMP BOSS GNU/Linux

Affected Component

Be precise.

Examples: '/etc/pam.d/common-auth — PAM authentication configuration' or

'/var/log/auth.log — Authentication log file permissions' or

'/usr/local/customtool — Unnecessary SUID binary'

Vulnerability Description

2-4 sentences. What is wrong + why it matters.

Example: 'The BOSS OS system does not implement an account lockout policy. The /etc/pam.d/common-auth configuration contains no pam_tally2 or pam_faillock module entries. This allows an attacker to make unlimited login attempts via SSH without being blocked, making brute-force password attacks trivially easy to execute against government user accounts.'

Steps to Reproduce

Numbered steps anyone can follow:

1. Open a terminal on BOSS OS system
2. Run: `cat /etc/pam.d/common-auth`
3. Observe no pam_tally2 or pam_faillock line is present
4. From another machine, run: `ssh wrongpassword@` repeatedly
5. Observe no lockout occurs after multiple failed attempts

Proof of Concept

Paste the exact command AND its output.

```
$ cat /etc/pam.d/common-auth
```

```
auth [success=1 default=ignore] pam_unix.so nullok
```

```
auth requisite pam_deny.so
```

(No pam_tally2 or pam_faillock line = no lockout configured)

Impact Assessment

What can an attacker actually DO?

Example: 'An attacker can script unlimited SSH login attempts against any user account. Using a password list, they can compromise accounts within hours. On a government system handling sensitive data, this could lead to unauthorized access to classified information and complete system compromise.'

Proposed Severity + Justification

Choose severity and explain WHY:

HIGH — because this directly enables brute-force attacks with no defense, and SSH is typically exposed. Any government system must have lockout protection.

Suggested Fix

Always include this — it shows expertise.

Add to /etc/pam.d/common-auth:

```
auth required pam_faillock.so preauth silent audit deny=5 unlock_time=900
```

This locks accounts after 5 failed attempts for 15 minutes.

CHAPTER 9

Master Cheat Sheet

Print this page. It is your entire competition plan on one page.

PS-02 Authentication — Quick Commands

```
cat /etc/shadow 2>/dev/null # CRITICAL if readable
sudo -l # What can I run as root?
grep 'NOPASSWD' /etc/sudoers # Passwordless sudo = CRITICAL
getent group sudo wheel admin # Who has admin access?
grep -v nologin /etc/passwd # All login-enabled accounts
cat /etc/pam.d/common-auth # Lockout policy?
ssh root@<BOSS_IP> # Root SSH allowed? (from Kali)
```

PS-07 File Permissions — Quick Commands

```
find / -perm -4000 -type f 2>/dev/null # All SUID files
find /etc /usr -perm -o+w -type f 2>/dev/null # World-writable system files
ls -la /etc/shadow /etc/passwd /etc/sudoers # Permission check
find / -name '*.bak' -o -name '*.old' 2>/dev/null # Backup files
find / -name 'id_rsa' 2>/dev/null # Private SSH keys
ls -la / | grep tmp # Check sticky bit
```

PS-08 Logging — Quick Commands

```
systemctl status auditd rsyslog # Are loggers running?
ls -la /var/log/ # What logs exist?
find /var/log -perm -o+w 2>/dev/null # World-writable logs = CRITICAL
auditctl -l 2>/dev/null # What is auditd watching?
cat /var/log/auth.log | tail -10 # Recent auth events
grep 'Failed' /var/log/auth.log # Failed logins logged?
```

Severity Quick Reference

Severity	Points/Prize	Example from Your Domains
CRITICAL	10 pts / Rs.10k	/etc/shadow readable, NOPASSWD:ALL sudo, world-writable logs
HIGH	8 pts / Rs.5k	No account lockout, root SSH allowed, unnecessary SUID
MEDIUM	5 pts / Rs.2k	Weak password policy, missing audit rules, backup files exposed
LOW	2 pts	Minor misconfigs, informational findings

Your AI Prompt When Stuck

■ AI PROMPT — Copy this into Claude/ChatGPT

I am participating in BOSS OS Bug Bounty 2026 (PS-02/PS-07/PS-08). I am testing a Debian-based Linux system. I need a Python/bash script to check [DESCRIBE WHAT YOU WANT TO CHECK]. Make it beginner friendly with clear output labels (CRITICAL/HIGH/MEDIUM/PASS). Include the exact commands and what good vs bad output looks like.

You are the strategist, Midhilesh.
You have Kali Linux. You have AI. You have this guide.
You do not need to know every command — you need to know what to ask.
Direct the tools. Read the results. Write the reports. Win.

Team ZeroDay · BOSS OS Bug Bounty 2026 · Go win it.